Лаба по флоатам

Цель данной лабораторной работы – своими руками посмотреть, какие подводные камни скрываются под стандартным форматом чисел с плавающей точкой. Она не столько о приобретении практического навыка, сколько о фундаментальном понимании того, что происходит, когда вы пытаетесь на современных компьютерах, телефонах или маках считать вещественные числа. Причем вне зависимости от языка, это именно особенность внутреннего устройства процессора. Там появляется немало проблем, с которыми все время приходится сталкиваться в реальной жизни научного программиста или инженера. В рамках первого курса вы с ними вряд ли столкнетесь (особо везучие и талантливые, конечно, поймают их при написании семестровых проектов), а вот потом…

Теоретически, есть вариант банально заучить все рецепты вроде «не сравнивайте float’ы с помощью ==» или «проводите все расчеты около 1», но не все так просто. Рецепты устаревают, но это было бы еще полбеды. Плохо то, что они иногда внезапно устаревают, вот в чем фокус. Поэтому сегодня вам нужно разобраться, откуда эти рецепты появляются, и какие новости в мире техники могут заставить вас эти рецепты пересмотреть.

Отработка полезных практических навыков в этой лабе тоже, впрочем, есть. Вам нужно будет крутить и пристраивать в разные места одни и те же кусочки своего кода, гуглить и строить графики по собственноручно насчитанным данным. Постарайтесь сразу организовывать код удобно, минимизируя ручной труд. Несколько подсказок для дополнительного удобства дадим в примерах кода, также не забывайте примеры из предыдущей лабораторной.

Первая арка (Анатомия float)

Для начала, давайте разберемся, как float устроен внутри и почему полезно иметь в голове схему его анатомии.

0. Немного черной магии своими руками

Как многие уже в курсе, в STL C++ есть стандартная функция для возведения в степень, std::pow. Она оперирует только числами с плавающей точкой, и если ей нужно иметь дело с целыми числами, то она их приводит к типу double. Обычно все проходит хорошо, но бывают нюансы. Давайте попробуем их нащупать.

Напишите свою функцию

```
unsigned long long int my_pow(unsigned long long int x,
                              unsigned long long int n)
```

, которая возводит число в степень (тип сразу взяли побольше, потому что нам понадобятся большие значения).

Выберите основание степени и посмотрите, как будут отличаться результаты работы вашей функции и стандартной для float и для double. Обратите внимание, что компилятор приводит целые числа именно к типу double, не просто float. Чтобы заставить его вызвать pow для float, нужно явно привести к нему оба аргумента. Вот пример кода, который заодно показывает, как вывести числа более-менее аккуратными столбиками. Первые две строчки меняют поведение cout и заставляют его выводить числа с плавающей точкой **нормально**. То есть не сжимая в формат со степенью и с постоянным количеством знаков после запятой. Можете их закомментировать и проверить, что будет.

```
cout << fixed;
cout.precision(2);
for (int i = 0; i < 100; i++)
    cout << i << "\t\t" << pow(3.0f, (float)i) << endl;
```

Поэкспериментируйте. Попробуйте разные значения степени и основания степени. Найдите пределы применимости стандартной функции pow для целых чисел.

Что нужно показать:

- таблица из трех столбцов: степень, результат работы my_pow для какого-то основания, результат работы std::pow для того же основания (float);
- таблица из трех столбцов: степень, результат работы my_pow для какого-то основания, результат работы std::pow для того же основания (double);

1. unsigned int -> binary

Давайте посмотрим, почему произошла магия из пункта 0. Для начала, сделайте промежуточный шаг, который понадобится дальше. Выведите на экран unsigned int в двоичной системе счисления, используя побитовые операции. Выводите все 32 бита, включая старшие нули. Так будет проще делать следующие пункты. Для тех же целей лучше сразу оформить этот кусочек кода как функцию.

Небольшие подсказки по алгоритму. Если вы уже и сами знаете, как делать, можете их игнорировать.

Вариант на минималках – использовать очевидный алгоритм с делением на степень двойки и взятием остатка, только вместо деления использовать побитовый сдвиг вправо (x >>= 1;), а вместо умножения – влево (x <<= 1;).

Вариант покруче – сдвиги туда-сюда. Обратите внимание, что при сдвиге, например, влево, справа появляются исключительно нули. То есть, если вы сдвинули число на 1 бит вправо, а потом на 1 бит влево, и оно после этого не изменилось, то у него в крайнем правом бите (который отвечает за младшую степень двойки, нулевую, то есть единицу) был ноль. Если число изменилось, то там была единица. Таким образом вы можете побитово обработать все число (в unsigned int этих битов 32). Обработали крайний бит, сдвинули на один бит вправо («откусили» обработанный бит), снова обработали крайний бит, снова сдвинули на один бит вправо. И так далее. Правда, в этом случае двоичные цифры будут идти в обратном порядке, и придется прибегать либо к массиву, либо к рекурсии. Но есть еще один вариант. Так как побитовым операциям все равно, в какую сторону гонять биты, вы можете обрабатывать число не справа налево, а слева направо, начиная с крайнего левого бита. Тогда двоичные цифры сразу будут в нужном вам порядке.

Вариант эксперта – битовые маски. Звучит страшно, а на деле легко. Здесь нужно воспользоваться побитовым «и» – одинарным &. Как это работает. Если взять число 1, то его побитовое представление будет 00…0001. Если сделать операцию «x & 1», то ее результатом будет единица или ноль – в зависимости от того, каким был младший бит у переменной x. А дальше как в предыдущем пункте. Обработали младший бит, сдвинули x вправо на один бит, снова обработали младший и так далее. Вместо сдвигания x можно сдвигать влево единицу, но тогда обратите внимание, что, например, результат «x & (1 << 3)» будет не 0 или 1, а 0 или 8. Так что если вместо ожидаемого 01001100 ваша программа выводит 064008400, то выводите не сам результат операции, а все же 0 или 1. Аналогично предыдущему случаю, число можно разбирать как с младших битов, так и со старших, чтобы не париться с порядком вывода.

Что нужно показать:

Ничего, это промежуточный пункт.

2. float -> binary

А вот и расшифровка магии, на которую можно посмотреть своими руками. Выведите на экран представление float в памяти компьютера. Проверьте, что IEEE 754-2008 (стандартный стандарт) действительно работает.

Первая сложность - заставить арифметические операции увидеть этот float как последовательность единиц и нулей. С unsigned int все было просто – целые положительные числа хранятся в двоичном разложении, и операции деления и взятия остатка у нас работают штатно. С float такое уже не прокатит. Взять остаток при делении вещественного числа на 0.125 – на такую экзотику разработчики архитектуры процессора не рассчитывали. То есть нужно посмотреть на float как на unsigned int, а потом использовать код из пункта 1 и вывести побитовое содержимое этого unsigned int, который мы записывали в память как float. Если просто присвоить float в unsigned int (чтобы воспользоваться функцией из пункта 1), то он округлится до целой части (можете сами проверить). Это не то, что нам нужно: нам нужно посмотреть на те же самые биты не как на float, а как на unsigned int.

Это можно сделать с помощью union (во второй половине семестра мы покажем более честный вариант с указателями, но пока что прилично сработает и с union). Он используется для создания новых пользовательских типов данных – как struct, и по синтаксису тоже очень похоже. Только в отличие от struct, у которого все поля независимы и лежат в памяти рядом, у union все поля зависимы и лежат в одном и том же месте памяти. То есть к одному и тому же участку памяти мы обращаемся разными способами – например, то как к float, то как к unsigned int. Это достаточно экзотическая конструкция языка, которая используется редко и не всегда законно, но в данном случае это ровно то, что нужно. Используйте union и код из пункта 1.

Когда получили загадочные последовательности 0 и 1 – постарайтесь их расшифровать и проверить, правда ли стандарт IEEE 754-2008 работает так, как про него рассказали на лекции. Попробуйте воспроизвести примеры оттуда и посмотреть на них со знанием двоичного разложения, которое реально выдает компьютер. Можно добавить пробелы медлу знаком и степенью, степенью и мантиссой. Так будет легче читать.

Что нужно показать:

- представление какого-либо вещественного числа в памяти компьютера.

3. Переполнение мантиссы

Вернемся к пункту 0 и черной магии. Вам нужно добавить четвертый столбец - представление соответствующего float/double в памяти компьютера. Так вы своими глазами увидите, что происходит в тот момент, когда стандартная функция начинает терять точность, и почему.

Что нужно показать:

- таблица из четырёх столбцов: степень, результат работы my_pow для какого-то основания, результат работы std::pow для того же основания (float), представление результата std::pow в памяти компьютера;
- таблица из четырёх столбцов: степень, результат работы my_pow для какого-то основания, результат работы std::pow для того же основания (double), представление результата std::pow в памяти компьютера;

4. Особые значения

Пока не ушли далеко от анатомии. У типа float/double есть несколько специальных значений: inf, -inf и nan. Подумайте, как их получить, и выведите на экран их битовое представление.

Что нужно показать:

- битовое представление inf, -inf и nan;

Вторая арка (Судмедэкспертиза float)

Помимо анатомии, которую не стоит выпускать из головы, также полезно помнить, что float - это конечный дискретный диапазон. Например, в этот диапазон не входит такое банальное число, как 0.2 - если вы попытаетесь сохранить это число, то вместо него в память сохранится какое-то другое, очень близкое, но не совсем то. При минимально сложных расчетах эта погрешность начинает сказываться. Давайте разберем этот эффект более подробно.

5. Бесконечный цикл

Самый простой способ ощутить, что диапазон float дискретный - это воспроизвести бесконечный цикл, возникающий неравномерности этого диапазона. Тут вам нужно воспользоваться тем фактом, что float'ы при увеличении своего значения начинают стоять все дальше и дальше друг от друга. Например, если вы будете использовать переменную типа float как счетчик цикла и пойдете с шагом 1.0 от нуля до очень большого числа, то рано или поздно дойдете до того места, где расстояние между соседними float'ами будет больше единицы. Тогда при прибавлении единицы к переменной цикла эта переменная вообще не изменится – мы как бы не смогли перепрыгнуть на следующее число, наш шаг потерялся в погрешности. И все, застряли в бесконечном цикле, который сами создали.

Что нужно показать:

- как ваш компьютер красиво застрял в бесконечном цикле и на каком значении счетчика это случилось.

6. Накопление ошибки

Просуммируйте массив вещественных чисел. Подберите значения так, чтобы сумма отличалась от ожидаемой.

Что нужно показать:

- как ваш компьютер красиво облажался в простом суммировании.

7. Неассоциативность

Модифицируйте предыдущий пример, изменив порядок операций. Подберите значения так, чтобы результат при разном порядке операций заметно отличался.

Что нужно показать:

- как ваш компьютер красиво облажался в более сложном суммировании.

8. Интегрировал, интегрировал да не выинтегрировал

Возьмите функцию, для которой можете аналитически посчитать площадь под графиком на отрезке от 0 до 1. Например, хорошо подойдет y = x, но можно взять и что-то более красивое. Посчитайте эту площадь методом прямоугольников (любым из них), сравните со значением, посчитанным аналитически.

Постройте графики самой функции и ее первообразной (так как первообразная определяется с точностью до константы, берите ту, которая удобнее влезает на график по масштабу). Постройте график зависимости значения площади от количества прямоугольников (размера шага интегрирования). Определите, какое количество прямоугольников дает оптимальный результат. Сравните, что будет при расчетах на типе float и на типе double.

Что нужно показать:

- график выбранной функции и ее первообразной;
- график зависимости значения площади от количества прямоугольников.

Выводы

Формат чисел с плавающей точкой обладает большим количеством объективных недостатков. Основное его достоинство - процессоры, которые его используют, относительно дешевы и доступны всем. Поэтому других вы особо не найдете, и придется смириться с тем, что даже в тривиальных случаях компьютер будет выдавать неточные результаты. Их даже на точное равенство проверять нельзя, почти любой расчет будет выполняться с погрешностью.

Нас спасает только то, что для инженерных целей точные значения обычно и не нужны. А чтобы погрешность не накапливалась, есть целый раздел прикладной математики - вычислительная математика. У вас она будет на третьем курсе, после матана и линала, и там вы сами увидите, что даже на этой неидеальной архитектуре можно получать практически полезные результаты.

А до этого просто не особо верьте калькуляторам.

Третья арка (Темные материи для тех, про пролистал до конца)

Поговорим про антипереполнение. Это сложная тема, которая требует очень хорошего интуитивного понимания, как устроен float, но в перспективе ее знание окупается. И задание в этой арке тоже более сложное, потому что вам придется самостоятельно подумать над подбором примеров для демонстрации эффектов.

Для начала как следует вспомните, что такое антипереполнение. Лучше всего взять ручку и бумажку и честно самостоятельно пройти по шагам пример с лекции.

После этого воспроизведите антипереполнение, подобрав пример самостоятельно. Посмотрите на представление денормализованных чисел в памяти.

Измерьте среднее время арифметических операций для обычных и денормализованных чисел.

Попробуйте ускорить ваш код из пункта 8 отключением денормализованных чисел и посмотреть, как это влияет на точность.

Разберитесь, чем отличаются DAZ и FTZ. Вам может пригодиться следующий код:

```
#include <xmmintrin.h>
    /* enable DAZ (denormals are zero) and FTZ (flush to zero) */
    _mm_setcsr(_mm_getcsr() | 0x8040);
    /* disable DAZ and FTZ */
    _mm_setcsr(_mm_getcsr() & ~0x8040);
```

Что нужно показать:

- как выглядят денормализованные числа в памяти;
- сравнение среднего времени операций с обычными и денормализованными числами;
- график ошибки расчета интеграла от времени работы алгоритма при наличии и отсутствии денормализованных чисел;
- график ошибки расчета интеграла от времени работы алгоритма при наличии денормализованных чисел в зависимости от состояния флагов DAZ и FTZ.